

COMP 3700
Software Modeling & Design

Chapter1: Unified Modeling Language
Dr. Xuechao Li



AUBURN

UNIVERSITY

SAMUEL GINN
COLLEGE OF ENGINEERING

Outline

- Introduction to object and class
- Introduction to UML
- Introduction to UML design

Introduction to UML

- What's UML
- Goals of UML
- UML diagrams



UML: Unified Modeling Language

- The Unified Modeling Language (UML) is an industry-standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems
 - Language: express idea, not a methodology
 - Modeling: Describing a software system at a high level of abstraction
 - Unified: UML has become a world standard

UML: Unified Modeling Language

- UML simplifies the process of software design, making a "blueprint" for construction

In Essence: a tool used through the analysis and design phases of system development for expressing the constructs and relationships of complex systems

Target Usage: for building object-oriented and component-based systems

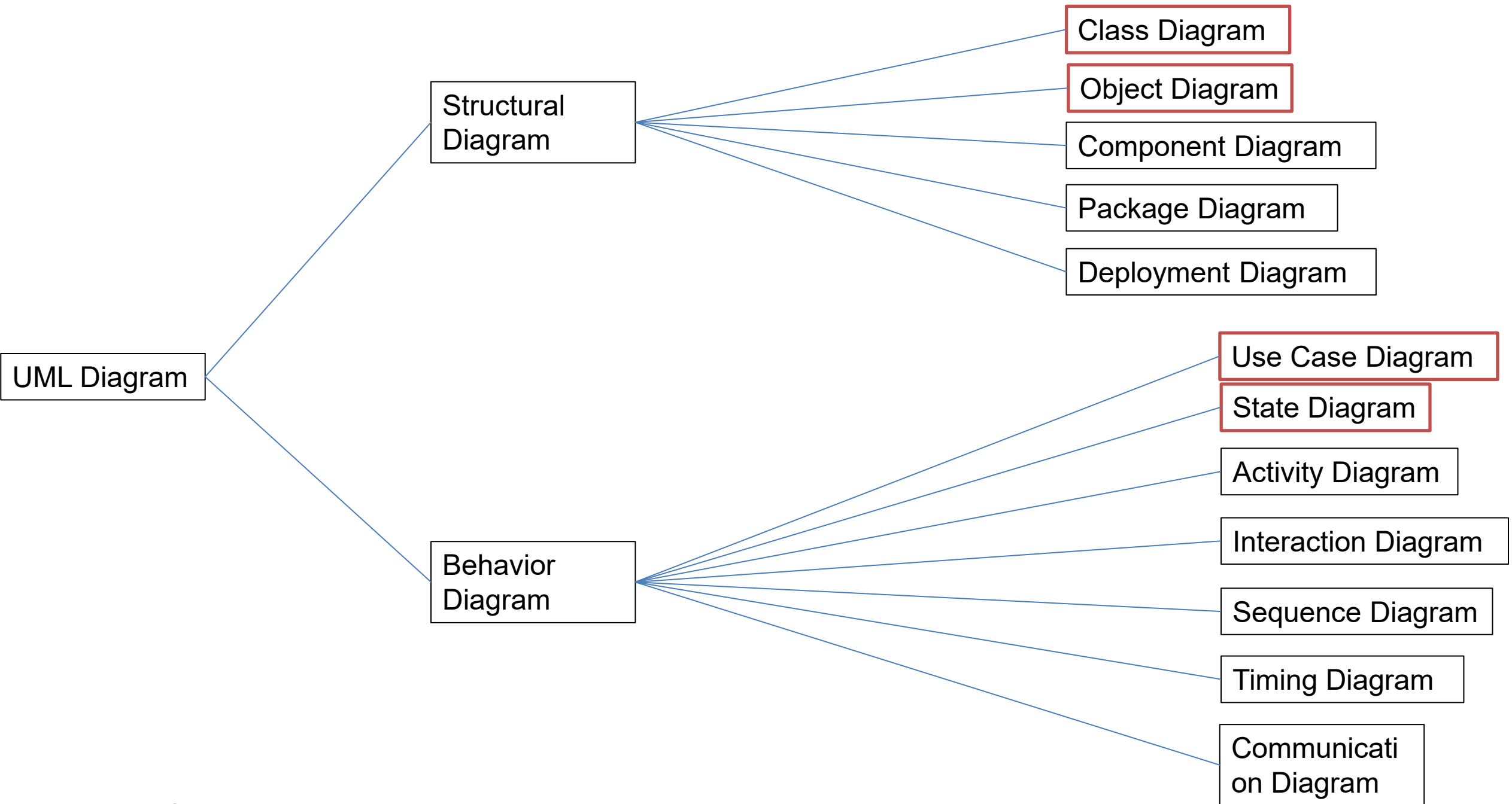
UML Overview

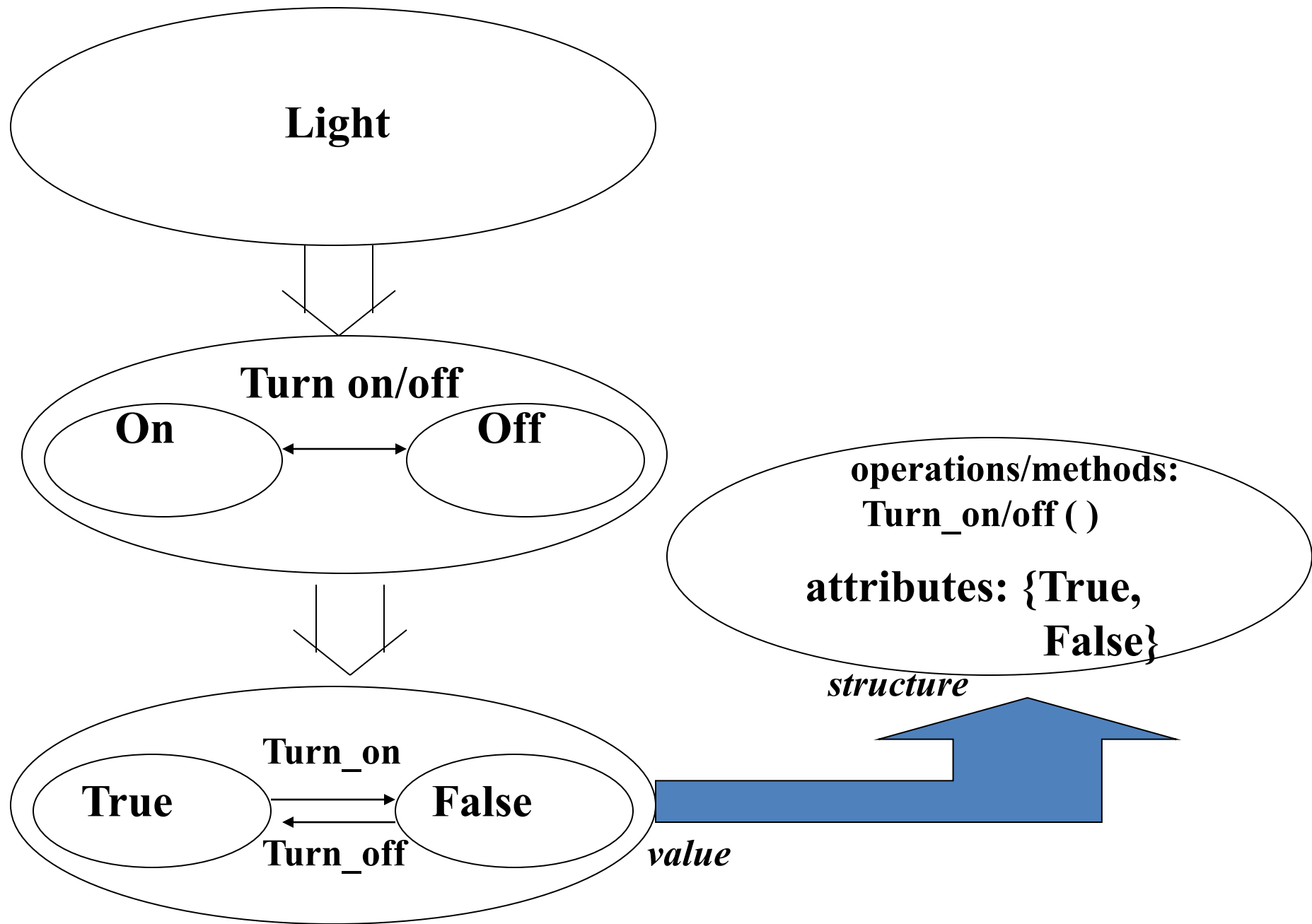
- UML is a language for visualizing, specifying, constructing and documenting the artifacts of a software system
- The artifacts might include requirements, architecture, design, source code, project plans, tests, prototypes, releases

Why we use UML?

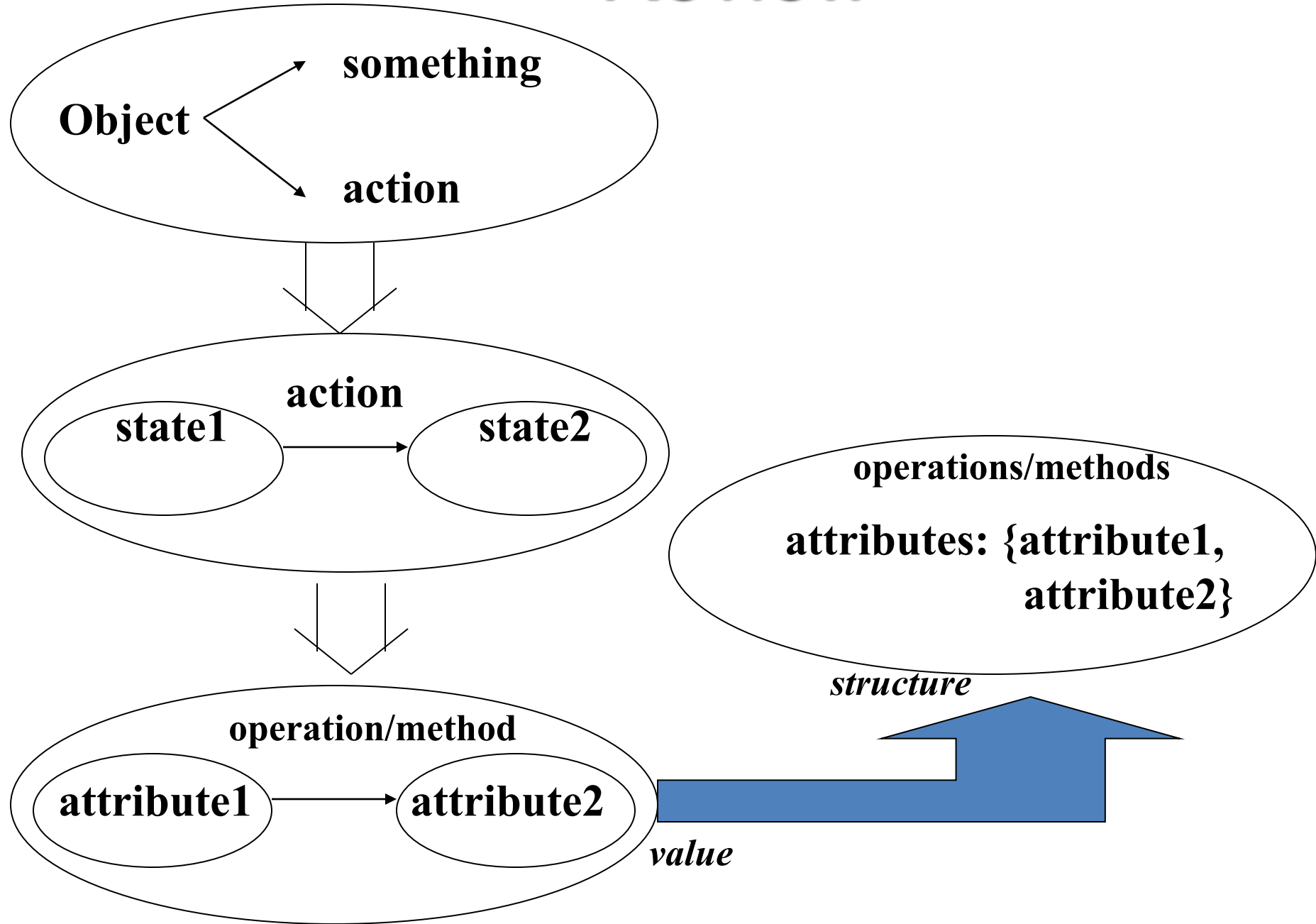
■ Why we use UML?

- Use graphical notation: more clearly than natural language (imprecise) and code (too detailed).
- Help acquire an overall view of a system.
- UML is *not* dependent on any one language or technology.
- UML moves us from fragmentation to standardization.





Review



Spot The Differences--1



Spot The Differences--2

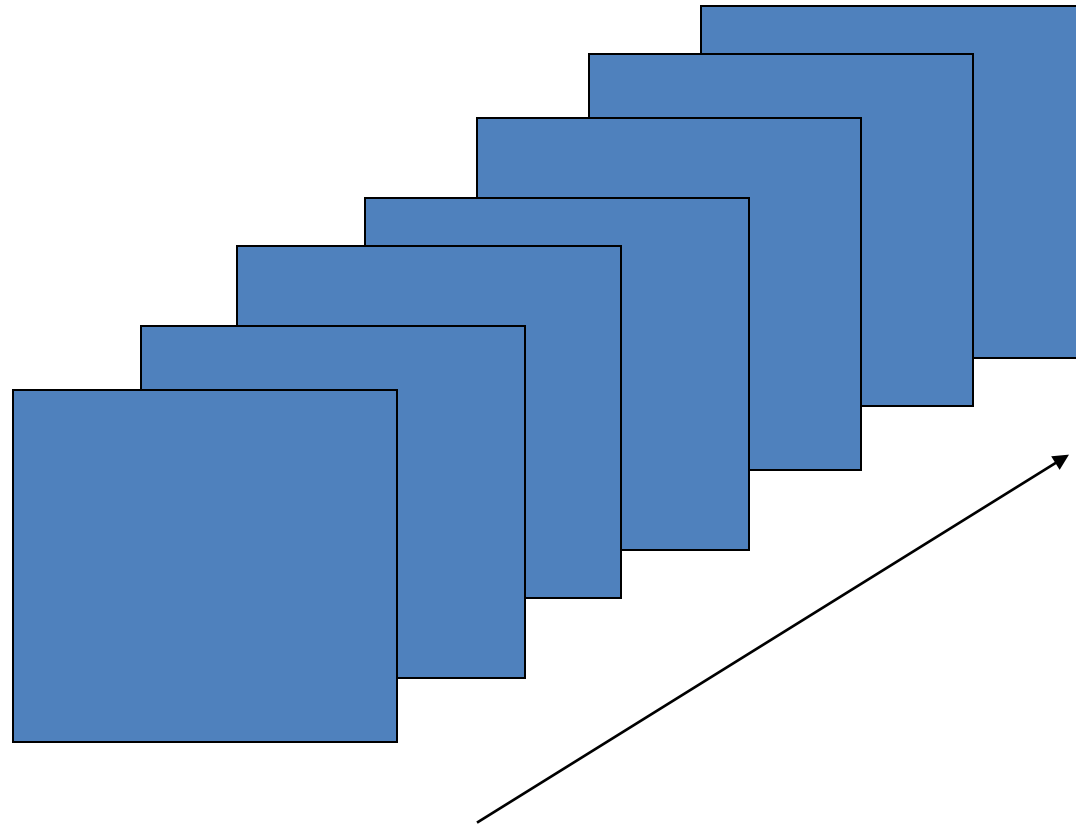


Introduction to object and class

- How to identify it from VERY similar pictures in the next slide
- Your partner will see the selected picture and other pictures together. Write down something to help your partner to pick up the same picture correctly

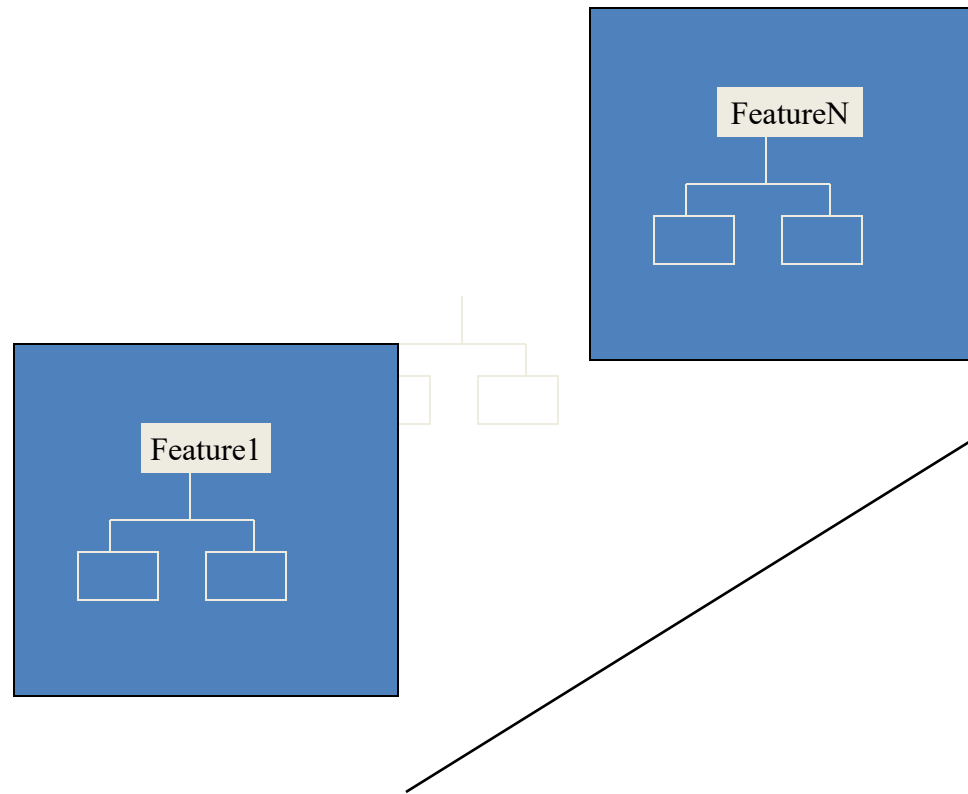
Introduction to object and class

- From snapshot to the action



Introduction to object and class

- From snapshot to the action



Introduction to object and class

- **What's object**
 - Identity
 - State
 - Behavior
 - Sequence Diagram
 - Statechart Diagram
 - Messages and methods
- **What's class**
 - Objects and Classes
 - Class Attributes
 - Operation (Method)
 - Interfaces
 - Interfaces and Implementation

What's object

- Conceptually, there are many ways to think of an object
 - something that can be seen or touched
 - a thing to which some action is directed
 - something that performs an action
- The structure and behaviour of similar objects are defined in their common class
- Objects have three properties: identity , state, and behaviour

Object Property 1: Identity

- Identity is that property of an object which distinguishes it from all other objects
- Most programming languages use variable names to refer to objects
- Keep in mind, however, that an object may not have a name; Similarly, an object might have multiple names (aliases)
 - For this reason, there is a subtle distinction made between the concepts of "name" and "identity"

Object Property 2: State

- The state of an object encompasses all of the descriptions of the object plus the current values of each of these descriptions

Object Property 3: Behaviour

- Behaviour is how an object acts and reacts, in terms of its state changes and message passing
- The state of an object represents the cumulative results of its behaviour
- In object-oriented programming, a behaviour is invoked by sending a message to an object

Example Objects

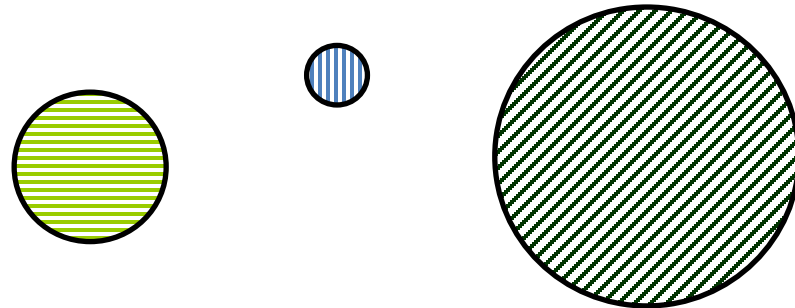
- There are many physical objects we can examine right in this room
 - each person is an object
 - any chair is not an object
 - each light bulb is an object
 - Any book is not an object
 - this room itself is an object (full or not)

What's the relationship among objects: Messages and Methods

- An object executes a method when it sends/receives a message
- For example, we can ask/tell a dog to sit by sending him the message "sit"
- In object-oriented programming, objects send messages asking/telling others to perform behaviors -- those other objects invoke the method corresponding to the message in order to execute the desired behavior

What's class

- A class describes the common structure (attributes/state) and behaviour of its instances
- For example,
 - 3.14, 2.71, and 5.5 can be classified as Floats
 - the following shapes can be classified as Circles



Classes and Objects

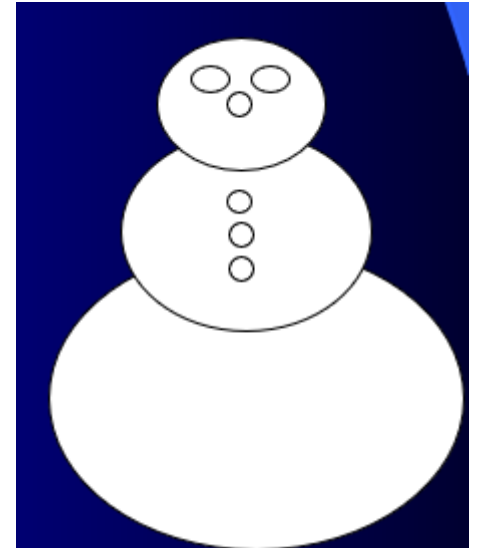
- An object is called an "instance" of a class
- The terms *instance* and *object* are interchangeable
- Creating an object from a class is often called *instantiation*
- For example, there are many person objects in this room -- each person is an instance of the person class

Classes and objects

- How would we describe the state, behaviour, and identity for each of these objects
- We have looked at objects and we have seen that objects can be "classified" into classes
- As programmers, we work with both classes and objects from those classes
- For example, we might write a stack class and create multiple stack objects for use in our program

- In a 2D drawing package, circles have a radius, a line thickness, a line colour, and a fill colour
- Each individual circle (instance) drawn by the user has its own value for each attribute
- The programmer writes a Circle class and the program instantiates a Circle object every time the user draws a Circle

a snowman made from 9
Circle instances



- Simply, a class is a group of objects with common parts or behaviors (similar, not necessary the exact same). That common thing will be defined in class definition.
- An object is an instance which meets all the description of its class.

Class Attributes

- An attribute is a named property of a class that describes the range of values that instances of the property may hold.(Booch,1999)
- An attribute has a type that defines the type of its instances.
- Only the object itself should be able to change the value of its attributes.
- The values of the attributes define the state of the object

Operation (Methods)

- An operation is the implementation of a service that can be requested from any object of the class to affect behavior (Booch, 1999)
- An operation can be:
 - Question (does not change the value of the object)
 - Command (may change the value of the object)

Interface and Implementation

- The class interface is its external (public) view
- The class implementation is its internal (private) view
- It is convenient to think of the interface as describing "what the objects of this class can do" and the implementation as "how the objects of this class do it"
- The implementation of a class consists of all the "behind the scenes" operations defined in the interface of the class